DBT TRAINING

Beginner · Module 02 · 60 min

Project Setup, Repo Structure & Execution Sequence

How dbt connects to Snowflake, what the project config controls, core CLI commands, and what actually happens when you run dbt.

`profiles.yml` — How dbt Connects to Snowflake

```yaml
analytics:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: abc123.eu-west-1
      user: jane@company.com
      authenticator: externalbrowser
      role: analytics_service_role
      warehouse: COMPUTE_WH_DEV
      database: SILVER_DEV
      schema: TESTING__dev_jane
      threads: 4

    prod:
      type: snowflake
      role: analytics_service_role
      warehouse: COMPUTE_WH_PROD
      database: SILVER
      schema: PUBLIC
      threads: 8
```

NOT in the repo. Lives at `~/.dbt/profiles.yml`. Contains credentials — never commit to git.

Dev schema rule

Your dev target writes to `TESTING.dev_{yourname}`. You cannot write to Silver or Gold from your laptop — that's the orchestrator's job.

`dbt debug` — run this first after cloning. Validates your connection before anything else.

`dbt_project.yml` — The Project Config

YAML — human-readable config. Indentation is everything: two spaces = one level.

```yaml
name: analytics
version: "1.0.0"
profile: analytics          # must match the key in profiles.yml

model-paths: ["models"]    # one or multiple folders to look for models
source-paths: ["sources"]   # one or multiple folders to look for sources

models:
  analytics:                # ← project namespace — must match name above
    staging:                # folder structure
      +materialized: view
      +tags: ["staging"]    # used for selective exectution
    silver:
      +materialized: table
      +tags: ["silver"]
      +persist_docs:       # ddl for comments and relationships
        relation: true
        columns: true
```

Why `analytics:`? It's a project namespace — scopes these configs to *your* models only, not to models from installed packages. Must match `name: analytics` at the top. Always include it.

`+database` and `+schema` — Where Models Land

`+materialized` controls *how* dbt builds a model. `+database` and `+schema` control *where* it lands in Snowflake.

```yaml
models:
  analytics:
    silver:
      +materialized: table
      +database: SILVER    # Silver models always write to this database
      +schema: PUBLIC

    gold:
      +materialized: table
      +database: GOLD      # Gold models write to a separate database
      +schema: PUBLIC
```

Without `+database`

Every layer lands in whatever database `profiles.yml` specifies. No separation.

With `+database`

Each layer is routed to its own Snowflake database. The separation you see in Snowflake is enforced here.

Core CLI Commands

Command	What it does	When to use
`dbt compile`	Renders Jinja → raw SQL, no execution	Inspecting compiled output
`dbt run`	Executes models, no tests	**In production - seperate tests**
`dbt test`	Runs tests only	Debugging one specific test
`dbt snapshot`	Creates snapshots	Update SCD2 models
`dbt build`	**All of the above**	Default in **development**
`dbt docs generate`	Builds doc site artifact	Regularly after new models

Remember: `dbt run` does NOT include snapshots. In production set up a separate task for `dbt snapshots` after bronze.

The Execution Sequence

What happens when you run `dbt run` or `dbt build`:

1. PARSE — Read all `.sql` and `.yml` files, validate Jinja syntax

Fails here: Jinja syntax errors, missing macro definitions

2. RESOLVE — Build the DAG — resolve all `{{ ref() }}` and `{{ source() }}` calls

Fails here: circular refs, missing models

3. COMPILE — Render Jinja → raw SQL, write to `target/compiled/`

Fails here: undefined variables, bad config blocks

4. EXECUTE — Send compiled SQL to Snowflake

Fails here: SQL errors, permission errors, type mismatches

5. REPORT — Log results, write `manifest.json` and `run_results.json`

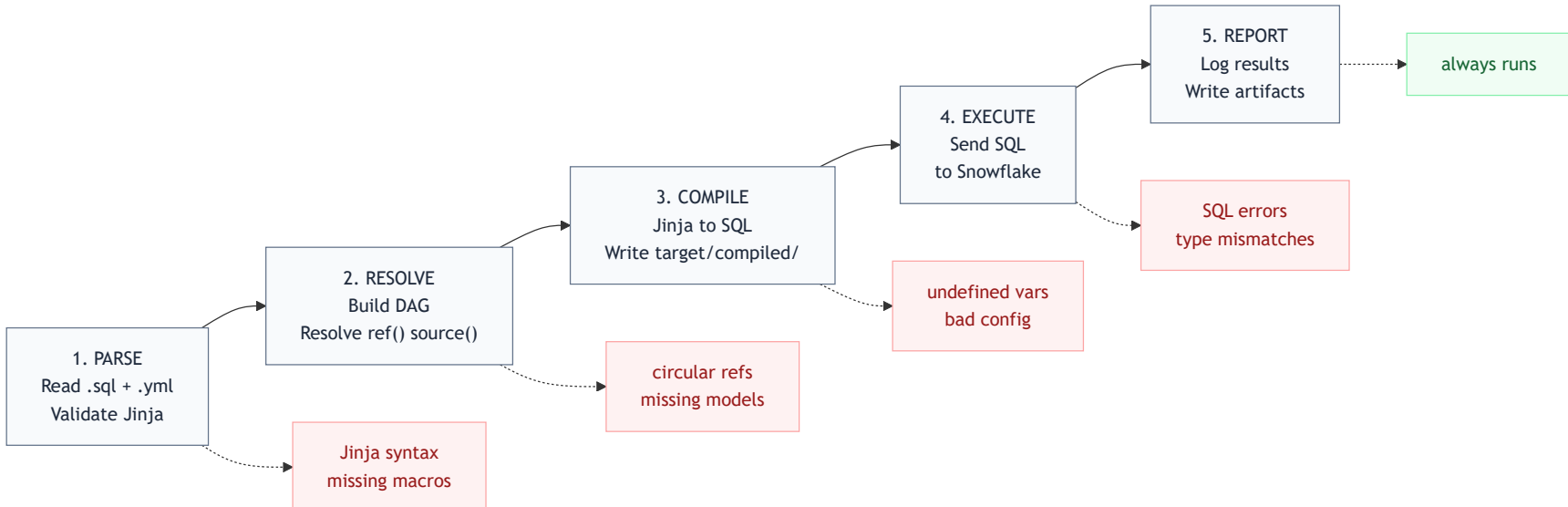
Always runs — even failed runs produce a report

LEGEND

- dbt only — no connection to the data warehouse
- First time connection to the data warehouse
- No effect on models — always runs regardless of outcome

Execution Sequence — Visualised

This gets very important when developing and debuging. When can a mistake happen and be caught.



Other Project Files You'll See

`packages.yml`

Declares external dbt packages (e.g. `dbt_utils`). Run `dbt deps` to install them. Use `dbt_utils` for surrogate key generation and date spines; `dbt_expectations` for extended test coverage.

`seeds/`

CSV files for small, static lookup tables that don't live in a source system — e.g. excluded test accounts, country-code mappings. Load with `dbt seed`.

`analyses/`

SQL files that use `ref()` and `source()` for lineage, but are never materialised. Useful for audit queries during a migration — compare old and new logic without polluting production models.

MODULE 02 COMPLETE

Next: Module 03

Jinja Basics for dbt

Prep Q1: Where does profiles.yml live — inside the repo or outside it?

Prep Q2: What does dbt build do that dbt run does not?

Prep Q3: At which phase does a Jinja syntax error appear?